

Sentiment Analysis on Textual Data (Movie Reviews)

Internship Task 4 - Natural Language Processing (NLP)

Author: AI Engineering Intern

Date: May 2026

Task Objectives

1. **Data Preprocessing:** Clean, tokenize, remove stopwords, and lemmatize textual data.
2. **Exploratory Data Analysis (EDA):** Analyze review lengths and visualize word patterns using Word Clouds.
3. **Feature Extraction:** Convert text data into numeric matrices using TF-IDF and Keras Word Embeddings.
4. **Model Implementation:**
 - **Lexicon-Based:** NLTK VADER (Valence Aware Dictionary and sEntiment Reasoner).
 - **Machine Learning:** Multinomial Naive Bayes and Logistic Regression.
 - **Deep Learning:** Recurrent Neural Network (LSTM) using TensorFlow/Keras.
5. **Evaluation:** Compare models using Accuracy, Precision, Recall, F1-score, Confusion Matrices, and ROC curves.
6. **Inference:** Build a prediction pipeline for unseen text.

1. Environment Setup & Library Imports

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import re
import string
import nltk
from nltk.corpus import movie_reviews, stopwords
from nltk.stem import WordNetLemmatizer
from nltk.sentiment.vader import SentimentIntensityAnalyzer
from wordcloud import WordCloud
from collections import Counter

# sklearn imports
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, accuracy_score, confusion_mat
```

```

# tensorflow imports
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout, SpatialDrop

# Set random seeds for reproducibility
np.random.seed(42)
tf.random.set_seed(42)

# Set visual styles
sns.set_theme(style='whitegrid', palette='muted')
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['font.size'] = 12

print("Libraries imported successfully!")
print("TensorFlow Version:", tf.__version__)
print("NLTK Version:", nltk.__version__)

```

Libraries imported successfully!
TensorFlow Version: 2.20.0
NLTK Version: 3.9.4

2. Load the Dataset

We use the NLTK `movie_reviews` dataset containing 2,000 movie reviews (1,000 positive and 1,000 negative).

```

In [2]: # Load positive and negative movie reviews into a pandas DataFrame
documents = []
for category in movie_reviews.categories():
    for fileid in movie_reviews.fileids(category):
        documents.append({
            'text': movie_reviews.raw(fileid),
            'sentiment': category
        })

df = pd.DataFrame(documents)
# Map pos/neg sentiment to binary labels (1/0)
df['label'] = df['sentiment'].map({'pos': 1, 'neg': 0})

print("Dataset loaded successfully!")
print("Total number of reviews:", len(df))
print("Class Distribution:\n", df['sentiment'].value_counts())
df.head()

```

Dataset loaded successfully!
Total number of reviews: 2000
Class Distribution:
sentiment
neg 1000
pos 1000
Name: count, dtype: int64

Out[2]:

	text	sentiment	label
0	plot : two teen couples go to a church party ,...	neg	0
1	the happy bastard's quick movie review \ndamn ...	neg	0
2	it is movies like these that make a jaded movi...	neg	0
3	" quest for camelot " is warner bros . ' firs...	neg	0
4	synopsis : a mentally unstable man undergoing ...	neg	0

3. Exploratory Data Analysis & Text Statistics

Let's compute statistics on review character counts and word counts to understand their scale.

```
In [3]: # Compute word and character counts for each review
df['char_count'] = df['text'].apply(len)
df['word_count'] = df['text'].apply(lambda x: len(x.split()))

print("Summary Statistics for Word Count:")
print(df['word_count'].describe())

# Plot review length distribution
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

sns.histplot(data=df, x='word_count', hue='sentiment', kde=True, bins=30, ax=axes[0])
axes[0].set_title('Review Word Count Distribution')
axes[0].set_xlabel('Word Count')
axes[0].set_ylabel('Frequency')

sns.boxplot(data=df, x='sentiment', y='word_count', ax=axes[1], palette={'pos':
axes[1].set_title('Review Word Count by Sentiment')
axes[1].set_xlabel('Sentiment')
axes[1].set_ylabel('Word Count')

plt.tight_layout()
plt.show()
```

Summary Statistics for Word Count:

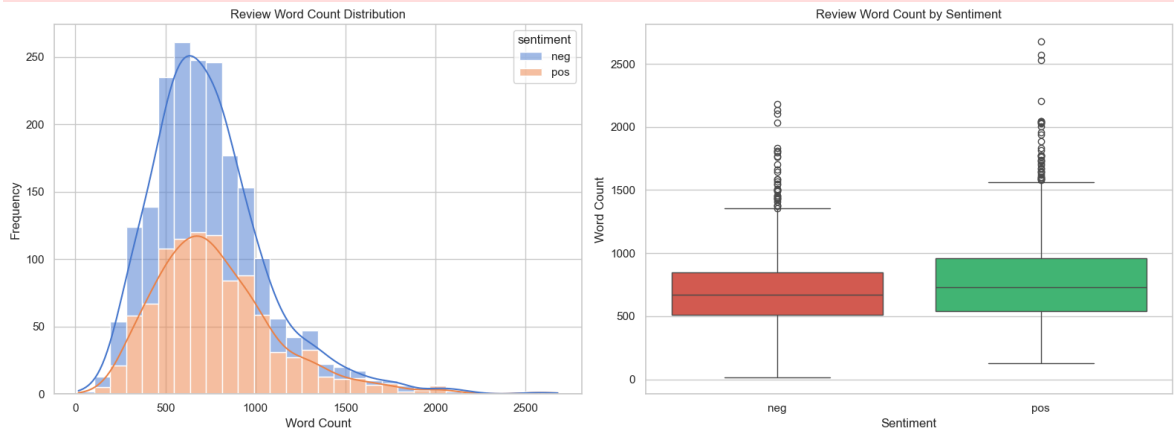
```
count    2000.00000
mean      746.34050
std       328.42529
min        17.00000
25%       526.75000
50%       696.50000
75%       904.00000
max      2678.00000
```

Name: word_count, dtype: float64

```
C:\Users\purva\AppData\Local\Temp\ipykernel_42140\3975751697.py:16: FutureWarnin
g:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v 0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.boxplot(data=df, x='sentiment', y='word_count', ax=axes[1], palette={'pos': '#2ecc71', 'neg': '#e74c3c'})
```



Let's look at a raw review sample to identify typical noise such as contractions, punctuation, and filler words.

```
In [4]: print("Raw review sample (first 400 characters):\n")
print(df['text'].iloc[0][:400] + "...")
```

Raw review sample (first 400 characters):

plot : two teen couples go to a church party , drink and then drive .
they get into an accident .
one of the guys dies , but his girlfriend continues to see him in her life , and
has nightmares .
what's the deal ?
watch the movie and " sorta " find out . . .
critique : a mind-fuck movie for the teen generation that touches on a very cool
idea , but presents it in a very bad package .
which i...

4. Text Preprocessing

To prepare raw text for modeling, we will run it through a standard cleaning pipeline:

1. Convert all characters to lowercase.
2. Strip any HTML tags or URLs.
3. Remove email addresses.
4. Filter out numbers, punctuation, and special characters.
5. Split the text into tokens (words).
6. Filter out standard English stop words and short words (length <= 2).
7. Lemmatize tokens to their base form (using NLTK's `WordNetLemmatizer`).

```
In [5]: stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()
```

```

def preprocess_text(text):
    # 1. Lowercase
    text = text.lower()

    # 2. Remove HTML tags
    text = re.sub(r'<[^>]*>', '', text)

    # 3. Remove URLs
    text = re.sub(r'https?://\S+|www\.\S+', '', text)

    # 4. Remove emails
    text = re.sub(r'\S+@\S+', '', text)

    # 5. Remove numbers and special characters/punctuation
    text = re.sub(r'^a-zA-Z\s', '', text)

    # 6. Tokenize & remove stopwords & lemmatize
    words = text.split()
    cleaned_words = [lemmatizer.lemmatize(word) for word in words if word not in

    return " ".join(cleaned_words)

# Apply preprocessing
print("Cleaning text data. Please wait...")
df['clean_text'] = df['text'].apply(preprocess_text)
print("Preprocessing completed!")

# Print comparison
print("\n--- Preprocessing Comparison ---")
print("Original:")
print(df['text'].iloc[0][:200] + "...")
print("\nCleaned:")
print(df['clean_text'].iloc[0][:200] + "...")

```

Cleaning text data. Please wait...

Preprocessing completed!

--- Preprocessing Comparison ---

Original:

plot : two teen couples go to a church party , drink and then drive .

they get into an accident .

one of the guys dies , but his girlfriend continues to see him in her life , and has nightmares .

w...

Cleaned:

plot two teen couple church party drink drive get accident one guy dy girlfriend continues see life nightmare whats deal watch movie sorta find critique mindfuck movie teen generation touch cool idea ...

5. Visual Insights: Word Clouds & Common Words

Visualizing positive and negative sentiment words helps us discover sentiment-bearing expressions.

```

In [6]: # Generate Word Clouds
pos_text = " ".join(df[df['label'] == 1]['clean_text'])
neg_text = " ".join(df[df['label'] == 0]['clean_text'])

```


C:\Users\purva\AppData\Local\Temp\ipykernel_42140\55996303.py:12: FutureWarning:

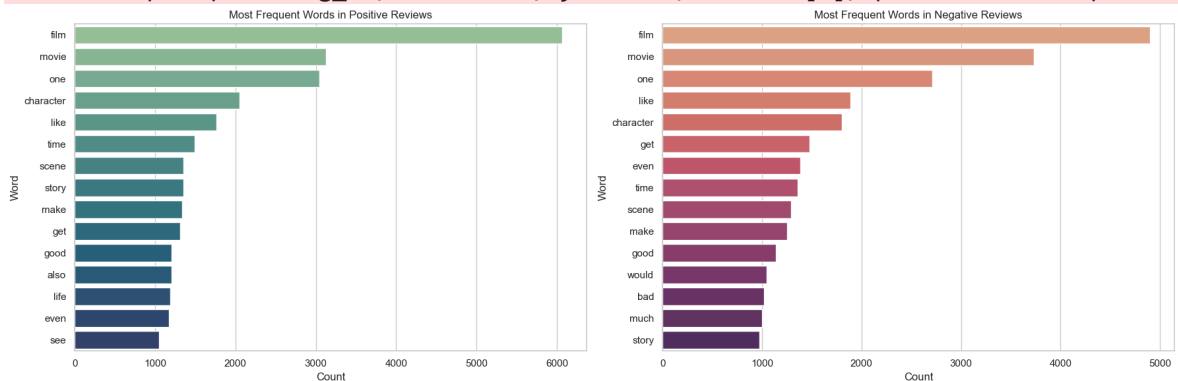
Passing `palette` without assigning `hue` is deprecated and will be removed in v 0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(data=pos_df, x='Count', y='Word', ax=axes[0], palette='crest')
```

C:\Users\purva\AppData\Local\Temp\ipykernel_42140\55996303.py:17: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v 0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(data=neg_df, x='Count', y='Word', ax=axes[1], palette='flare')
```



6. Train-Test Split

We divide the dataset into 80% training (1,600 samples) and 20% testing (400 samples) using stratification.

```
In [8]: X_train, X_test, y_train, y_test = train_test_split(
        df['clean_text'],
        df['label'],
        test_size=0.2,
        random_state=42,
        stratify=df['label']
    )

print(f"Training set size: {X_train.shape[0]}")
print(f"Testing set size: {X_test.shape[0]}")
```

Training set size: 1600

Testing set size: 400

7. Model 1: Lexicon-Based Sentiment Analysis (NLTK VADER)

NLTK VADER calculates sentiment polarity scores using an unsupervised lookup approach. We run VADER on the original RAW text because punctuation (like '!') and casing (like 'GREAT') affect its scores.

```
In [9]: # Initialize VADER Sentiment Intensity Analyzer
sia = SentimentIntensityAnalyzer()
```

```

# Run predictions
print("Running VADER predictions...")
vader_preds = []
for text in df.loc[X_test.index, 'text']:
    scores = sia.polarity_scores(text)
    # Compound score >= 0.05 is positive, else negative
    vader_preds.append(1 if scores['compound'] >= 0.05 else 0)

# Evaluate VADER
vader_acc = accuracy_score(y_test, vader_preds)
print(f"VADER Accuracy: {vader_acc:.4f}\n")
print("VADER Classification Report:")
print(classification_report(y_test, vader_preds))

```

Running VADER predictions...

VADER Accuracy: 0.6475

VADER Classification Report:

	precision	recall	f1-score	support
0	0.72	0.48	0.58	200
1	0.61	0.81	0.70	200
accuracy			0.65	400
macro avg	0.66	0.65	0.64	400
weighted avg	0.66	0.65	0.64	400

8. Feature Extraction for Machine Learning (TF-IDF)

We convert cleaned text to a numerical matrix using TF-IDF Vectorization, considering unigrams and bigrams up to a maximum of 5,000 features.

```

In [10]: vectorizer = TfidfVectorizer(max_features=5000, ngram_range=(1, 2))

X_train_tfidf = vectorizer.fit_transform(X_train)
X_test_tfidf = vectorizer.transform(X_test)

print("TF-IDF Matrix shapes:")
print("X_train_tfidf:", X_train_tfidf.shape)
print("X_test_tfidf:", X_test_tfidf.shape)

```

TF-IDF Matrix shapes:

X_train_tfidf: (1600, 5000)

X_test_tfidf: (400, 5000)

9. Model 2: Multinomial Naive Bayes

Naive Bayes calculates probability based on word counts. It's a standard, highly efficient model for text classification.

```

In [11]: # Train Naive Bayes
nb_model = MultinomialNB()
nb_model.fit(X_train_tfidf, y_train)

```



```

# Predict
nb_preds = nb_model.predict(X_test_tfidf)
nb_probs = nb_model.predict_proba(X_test_tfidf)[:, 1]

# Evaluate
nb_acc = accuracy_score(y_test, nb_preds)
print(f"Naive Bayes Accuracy: {nb_acc:.4f}\n")
print("Naive Bayes Classification Report:")
print(classification_report(y_test, nb_preds))

```

Naive Bayes Accuracy: 0.8225

Naive Bayes Classification Report:

	precision	recall	f1-score	support
0	0.81	0.84	0.83	200
1	0.84	0.80	0.82	200
accuracy			0.82	400
macro avg	0.82	0.82	0.82	400
weighted avg	0.82	0.82	0.82	400

10. Model 3: Logistic Regression

Logistic Regression works exceptionally well with high-dimensional sparse TF-IDF matrices.

```

In [12]: # Train Logistic Regression
lr_model = LogisticRegression(max_iter=1000)
lr_model.fit(X_train_tfidf, y_train)

# Predict
lr_preds = lr_model.predict(X_test_tfidf)
lr_probs = lr_model.predict_proba(X_test_tfidf)[:, 1]

# Evaluate
lr_acc = accuracy_score(y_test, lr_preds)
print(f"Logistic Regression Accuracy: {lr_acc:.4f}\n")
print("Logistic Regression Classification Report:")
print(classification_report(y_test, lr_preds))

```

Logistic Regression Accuracy: 0.8350

Logistic Regression Classification Report:

	precision	recall	f1-score	support
0	0.85	0.82	0.83	200
1	0.83	0.85	0.84	200
accuracy			0.83	400
macro avg	0.84	0.83	0.83	400
weighted avg	0.84	0.83	0.83	400

11. Model 4: Deep Learning (Recurrent Neural Network - LSTM)

Unlike TF-IDF which loses word ordering, Recurrent Neural Networks (like LSTM) can process sequential context. First, we map words to unique integers, and pad the reviews to a uniform length of 200 words.

```
In [13]: # Tokenization and Padding for Deep Learning
max_words = 10000
max_len = 200

dl_tokenizer = Tokenizer(num_words=max_words)
dl_tokenizer.fit_on_texts(X_train)

X_train_seq = pad_sequences(dl_tokenizer.texts_to_sequences(X_train), maxlen=max_len)
X_test_seq = pad_sequences(dl_tokenizer.texts_to_sequences(X_test), maxlen=max_len)

print("Sequences shape:")
print("X_train_seq:", X_train_seq.shape)
print("X_test_seq:", X_test_seq.shape)
```

```
Sequences shape:
X_train_seq: (1600, 200)
X_test_seq: (400, 200)
```

We construct a model with an `Embedding` layer (learning 128-dimensional word vectors), a `SpatialDropout1D` for regularization, an `LSTM` layer with 64 units, and a sigmoid outputs layer.

```
In [14]: # Model Architecture
lstm_model = Sequential([
    Embedding(input_dim=max_words, output_dim=128, input_length=max_len),
    SpatialDropout1D(0.2),
    LSTM(64, dropout=0.2),
    Dense(1, activation='sigmoid')
])

lstm_model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
lstm_model.summary()
```

```
C:\Users\purva\AppData\Local\Programs\Python\Python313\Lib\site-packages\keras\src\layers\core\embedding.py:97: UserWarning: Argument `input_length` is deprecated. Just remove it.
warnings.warn(
```

Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	?	0 (unbuilt)
spatial_dropout1d (SpatialDropout1D)	?	0
lstm (LSTM)	?	0 (unbuilt)
dense (Dense)	?	0 (unbuilt)



Total params: 0 (0.00 B)

Trainable params: 0 (0.00 B)

Non-trainable params: 0 (0.00 B)

We train the model for 5 epochs using a batch size of 64 and a validation split of 10%.

```
In [15]: # Train LSTM model
history = lstm_model.fit(
    X_train_seq, y_train,
    epochs=5,
    batch_size=64,
    validation_split=0.1,
    verbose=1
)
```

Epoch 1/5

1/23	54s	2s/step	- accuracy: 0.5781	- loss: 0.6936
2/23	2s	117ms/step	- accuracy: 0.5352	- loss: 0.6937
3/23	2s	119ms/step	- accuracy: 0.5269	- loss: 0.6936
4/23	2s	125ms/step	- accuracy: 0.5241	- loss: 0.6935
5/23	2s	125ms/step	- accuracy: 0.5255	- loss: 0.6933
6/23	2s	125ms/step	- accuracy: 0.5239	- loss: 0.6933
7/23	2s	125ms/step	- accuracy: 0.5227	- loss: 0.6932
8/23	1s	127ms/step	- accuracy: 0.5230	- loss: 0.6932
9/23	1s	129ms/step	- accuracy: 0.5236	- loss: 0.6931
10/23	1s	129ms/step	- accuracy: 0.5243	- loss: 0.6931
11/23	1s	130ms/step	- accuracy: 0.5241	- loss: 0.6930
12/23	1s	133ms/step	- accuracy: 0.5242	- loss: 0.6930
13/23	1s	135ms/step	- accuracy: 0.5243	- loss: 0.6930
14/23	1s	137ms/step	- accuracy: 0.5245	- loss: 0.6929
15/23	1s	138ms/step	- accuracy: 0.5246	- loss: 0.6929
16/23	0s	137ms/step	- accuracy: 0.5250	- loss: 0.6929
17/23	0s	135ms/step	- accuracy: 0.5254	- loss: 0.6928
18/23	0s	135ms/step	- accuracy: 0.5261	- loss: 0.6928
19/23	0s	134ms/step	- accuracy: 0.5267	- loss: 0.6927
20/23	0s	133ms/step	- accuracy: 0.5273	- loss: 0.6927
21/23	0s	134ms/step	- accuracy: 0.5280	- loss: 0.6926
22/23	0s	133ms/step	- accuracy: 0.5288	- loss: 0.6926

```

23/23 ----- 0s 132ms/step - accuracy: 0.5296 - loss: 0.6925
23/23 ----- 6s 159ms/step - accuracy: 0.5472 - loss: 0.6914 - val_
accuracy: 0.5375 - val_loss: 0.6902
Epoch 2/5
1/23 ----- 3s 150ms/step - accuracy: 0.8125 - loss: 0.6677
2/23 ----- 2s 108ms/step - accuracy: 0.7969 - loss: 0.6683
3/23 ----- 2s 111ms/step - accuracy: 0.7899 - loss: 0.6681
4/23 ----- 2s 108ms/step - accuracy: 0.7839 - loss: 0.6676
5/23 ----- 1s 109ms/step - accuracy: 0.7771 - loss: 0.6671
6/23 ----- 1s 110ms/step - accuracy: 0.7743 - loss: 0.6663
7/23 ----- 1s 113ms/step - accuracy: 0.7740 - loss: 0.6650
8/23 ----- 1s 116ms/step - accuracy: 0.7732 - loss: 0.6636
9/23 ----- 1s 117ms/step - accuracy: 0.7722 - loss: 0.6618
10/23 ----- 1s 118ms/step - accuracy: 0.7711 - loss: 0.6594
11/23 ----- 1s 118ms/step - accuracy: 0.7697 - loss: 0.6572
12/23 ----- 1s 120ms/step - accuracy: 0.7685 - loss: 0.6551
13/23 ----- 1s 121ms/step - accuracy: 0.7673 - loss: 0.6531
14/23 ----- 1s 121ms/step - accuracy: 0.7665 - loss: 0.6511
15/23 ----- 0s 121ms/step - accuracy: 0.7656 - loss: 0.6491
16/23 ----- 0s 121ms/step - accuracy: 0.7646 - loss: 0.6474
17/23 ----- 0s 120ms/step - accuracy: 0.7633 - loss: 0.6458
18/23 ----- 0s 121ms/step - accuracy: 0.7621 - loss: 0.6443
19/23 ----- 0s 121ms/step - accuracy: 0.7608 - loss: 0.6430
20/23 ----- 0s 120ms/step - accuracy: 0.7593 - loss: 0.6418
21/23 ----- 0s 120ms/step - accuracy: 0.7578 - loss: 0.6407
22/23 ----- 0s 120ms/step - accuracy: 0.7565 - loss: 0.6396
23/23 ----- 0s 119ms/step - accuracy: 0.7553 - loss: 0.6385
23/23 ----- 3s 130ms/step - accuracy: 0.7306 - loss: 0.6153 - val_
accuracy: 0.6875 - val_loss: 0.5953
Epoch 3/5
1/23 ----- 4s 191ms/step - accuracy: 0.8438 - loss: 0.5082
2/23 ----- 2s 131ms/step - accuracy: 0.8398 - loss: 0.5137
3/23 ----- 2s 137ms/step - accuracy: 0.8411 - loss: 0.5127

```

```

4/23  ————— 2s 136ms/step - accuracy: 0.8467 - loss: 0.5101
5/23  ————— 2s 138ms/step - accuracy: 0.8517 - loss: 0.5073
6/23  ————— 2s 136ms/step - accuracy: 0.8543 - loss: 0.5047
7/23  ————— 2s 136ms/step - accuracy: 0.8566 - loss: 0.5007
8/23  ————— 2s 135ms/step - accuracy: 0.8599 - loss: 0.4947
9/23  ————— 1s 135ms/step - accuracy: 0.8637 - loss: 0.4877
10/23 ————— 1s 135ms/step - accuracy: 0.8673 - loss: 0.4803
11/23 ————— 1s 135ms/step - accuracy: 0.8707 - loss: 0.4728
12/23 ————— 1s 135ms/step - accuracy: 0.8741 - loss: 0.4647
13/23 ————— 1s 135ms/step - accuracy: 0.8771 - loss: 0.4570
14/23 ————— 1s 134ms/step - accuracy: 0.8787 - loss: 0.4509
15/23 ————— 1s 135ms/step - accuracy: 0.8799 - loss: 0.4453
16/23 ————— 0s 135ms/step - accuracy: 0.8808 - loss: 0.4403
17/23 ————— 0s 135ms/step - accuracy: 0.8816 - loss: 0.4357
18/23 ————— 0s 135ms/step - accuracy: 0.8824 - loss: 0.4311
19/23 ————— 0s 135ms/step - accuracy: 0.8831 - loss: 0.4268
20/23 ————— 0s 135ms/step - accuracy: 0.8838 - loss: 0.4229
21/23 ————— 0s 135ms/step - accuracy: 0.8846 - loss: 0.4191
22/23 ————— 0s 135ms/step - accuracy: 0.8854 - loss: 0.4154
23/23 ————— 0s 134ms/step - accuracy: 0.8863 - loss: 0.4118
23/23 ————— 3s 145ms/step - accuracy: 0.9049 - loss: 0.3330 - val_
accuracy: 0.8000 - val_loss: 0.4273
Epoch 4/5
1/23  ————— 3s 171ms/step - accuracy: 0.9219 - loss: 0.1895
2/23  ————— 2s 139ms/step - accuracy: 0.8945 - loss: 0.2206
3/23  ————— 2s 137ms/step - accuracy: 0.8950 - loss: 0.2223
4/23  ————— 2s 136ms/step - accuracy: 0.8988 - loss: 0.2195
5/23  ————— 2s 138ms/step - accuracy: 0.9034 - loss: 0.2152
6/23  ————— 2s 136ms/step - accuracy: 0.9078 - loss: 0.2109
7/23  ————— 2s 136ms/step - accuracy: 0.9117 - loss: 0.2062
8/23  ————— 2s 141ms/step - accuracy: 0.9157 - loss: 0.2012

```

```

9/23  ————— 1s 142ms/step - accuracy: 0.9194 - loss: 0.1965
10/23 ————— 1s 145ms/step - accuracy: 0.9222 - loss: 0.1928
11/23 ————— 1s 145ms/step - accuracy: 0.9249 - loss: 0.1891
12/23 ————— 1s 146ms/step - accuracy: 0.9272 - loss: 0.1857
13/23 ————— 1s 146ms/step - accuracy: 0.9293 - loss: 0.1824
14/23 ————— 1s 145ms/step - accuracy: 0.9309 - loss: 0.1796
15/23 ————— 1s 146ms/step - accuracy: 0.9322 - loss: 0.1772
16/23 ————— 1s 145ms/step - accuracy: 0.9330 - loss: 0.1758
17/23 ————— 0s 145ms/step - accuracy: 0.9338 - loss: 0.1742
18/23 ————— 0s 146ms/step - accuracy: 0.9346 - loss: 0.1726
19/23 ————— 0s 145ms/step - accuracy: 0.9354 - loss: 0.1711
20/23 ————— 0s 145ms/step - accuracy: 0.9361 - loss: 0.1695
21/23 ————— 0s 145ms/step - accuracy: 0.9369 - loss: 0.1680
22/23 ————— 0s 144ms/step - accuracy: 0.9378 - loss: 0.1664
23/23 ————— 0s 142ms/step - accuracy: 0.9386 - loss: 0.1650
23/23 ————— 4s 151ms/step - accuracy: 0.9563 - loss: 0.1331 - val_
accuracy: 0.7875 - val_loss: 0.5097
Epoch 5/5
1/23  ————— 3s 160ms/step - accuracy: 1.0000 - loss: 0.0918
2/23  ————— 2s 120ms/step - accuracy: 1.0000 - loss: 0.0920
3/23  ————— 2s 121ms/step - accuracy: 0.9983 - loss: 0.0921
4/23  ————— 2s 130ms/step - accuracy: 0.9977 - loss: 0.0907
5/23  ————— 2s 135ms/step - accuracy: 0.9976 - loss: 0.0886
6/23  ————— 2s 134ms/step - accuracy: 0.9975 - loss: 0.0864
7/23  ————— 2s 133ms/step - accuracy: 0.9976 - loss: 0.0843
8/23  ————— 1s 131ms/step - accuracy: 0.9976 - loss: 0.0820
9/23  ————— 1s 133ms/step - accuracy: 0.9977 - loss: 0.0798
10/23 ————— 1s 134ms/step - accuracy: 0.9978 - loss: 0.0775
11/23 ————— 1s 134ms/step - accuracy: 0.9978 - loss: 0.0754
12/23 ————— 1s 134ms/step - accuracy: 0.9979 - loss: 0.0734
13/23 ————— 1s 137ms/step - accuracy: 0.9980 - loss: 0.0716

```

```

#####
14/23 ----- 1s 137ms/step - accuracy: 0.9979 - loss: 0.0701
#####
15/23 ----- 1s 137ms/step - accuracy: 0.9977 - loss: 0.0688
#####
16/23 ----- 0s 137ms/step - accuracy: 0.9975 - loss: 0.0679
#####
17/23 ----- 0s 137ms/step - accuracy: 0.9973 - loss: 0.0670
#####
18/23 ----- 0s 136ms/step - accuracy: 0.9972 - loss: 0.0661
#####
19/23 ----- 0s 136ms/step - accuracy: 0.9970 - loss: 0.0654
#####
20/23 ----- 0s 136ms/step - accuracy: 0.9969 - loss: 0.0647
#####
21/23 ----- 0s 136ms/step - accuracy: 0.9967 - loss: 0.0640
#####
22/23 ----- 0s 136ms/step - accuracy: 0.9966 - loss: 0.0633
#####
23/23 ----- 0s 134ms/step - accuracy: 0.9965 - loss: 0.0626
#####
23/23 ----- 3s 144ms/step - accuracy: 0.9944 - loss: 0.0478 - val_
accuracy: 0.8062 - val_loss: 0.5561

```

Let's plot the training and validation accuracy and loss curves.

```

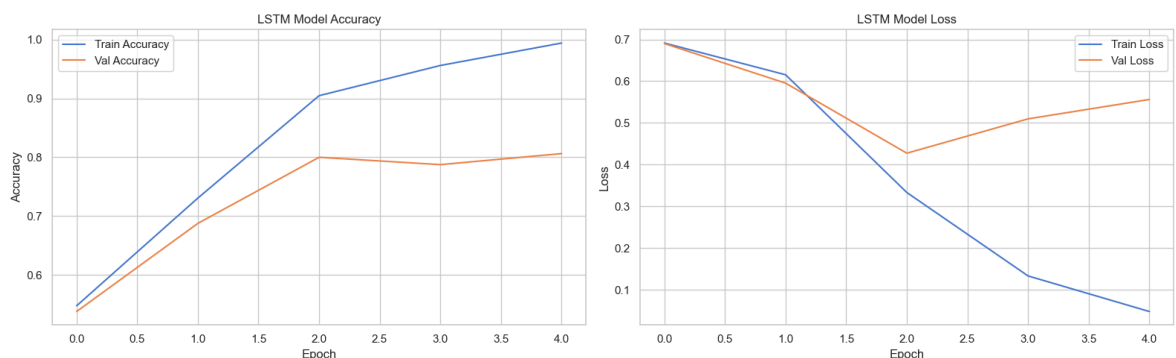
In [16]: # Plot training & validation accuracy/loss
fig, axes = plt.subplots(1, 2, figsize=(16, 5))

# Accuracy
axes[0].plot(history.history['accuracy'], label='Train Accuracy')
axes[0].plot(history.history['val_accuracy'], label='Val Accuracy')
axes[0].set_title('LSTM Model Accuracy')
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Accuracy')
axes[0].legend()

# Loss
axes[1].plot(history.history['loss'], label='Train Loss')
axes[1].plot(history.history['val_loss'], label='Val Loss')
axes[1].set_title('LSTM Model Loss')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Loss')
axes[1].legend()

plt.tight_layout()
plt.show()

```



Now, we evaluate the LSTM network's accuracy and generate a classification report on the testing set.

```
In [17]: # Predict probabilities
lstm_probs = lstm_model.predict(X_test_seq).flatten()
# Convert to labels
lstm_preds = (lstm_probs >= 0.5).astype(int)

# Evaluate
lstm_acc = accuracy_score(y_test, lstm_preds)
print(f"LSTM Accuracy: {lstm_acc:.4f}\n")
print("LSTM Classification Report:")
print(classification_report(y_test, lstm_preds))
```

```
1/13 ————— 2s 232ms/step
????????????????????????????????????????????????????????
4/13 ————— 0s 22ms/step
????????????????????????????????????????????????????????
7/13 ————— 0s 22ms/step
????????????????????????????????????????????????????????
10/13 ————— 0s 22ms/step
????????????????????????????????????????????????????????
13/13 ————— 0s 35ms/step
????????????????????????????????????????????????????????
13/13 ————— 1s 37ms/step
LSTM Accuracy: 0.8000
```

```
LSTM Classification Report:
              precision    recall  f1-score   support

     0       0.79       0.81       0.80       200
     1       0.81       0.79       0.80       200

 accuracy          0.80
 macro avg         0.80       0.80       0.80       400
weighted avg         0.80       0.80       0.80       400
```

12. Model Comparison & Analysis

We compare the metrics of our Lexicon-based baseline (VADER), Naive Bayes, Logistic Regression, and the LSTM Network.

```
In [18]: # Compute ROC-AUC
vader_auc = roc_auc_score(y_test, vader_preds)
nb_auc = roc_auc_score(y_test, nb_probs)
lr_auc = roc_auc_score(y_test, lr_probs)
lstm_auc = roc_auc_score(y_test, lstm_probs)

# Summary table
comparison_df = pd.DataFrame({
    'Model': ['NLTK VADER', 'Multinomial Naive Bayes', 'Logistic Regression', 'LSTM'],
    'Accuracy': [vader_acc, nb_acc, lr_acc, lstm_acc],
    'ROC-AUC': [vader_auc, nb_auc, lr_auc, lstm_auc]
})

print("--- Model Performance Summary ---")
```



```
print(comparison_df.to_string(index=False))

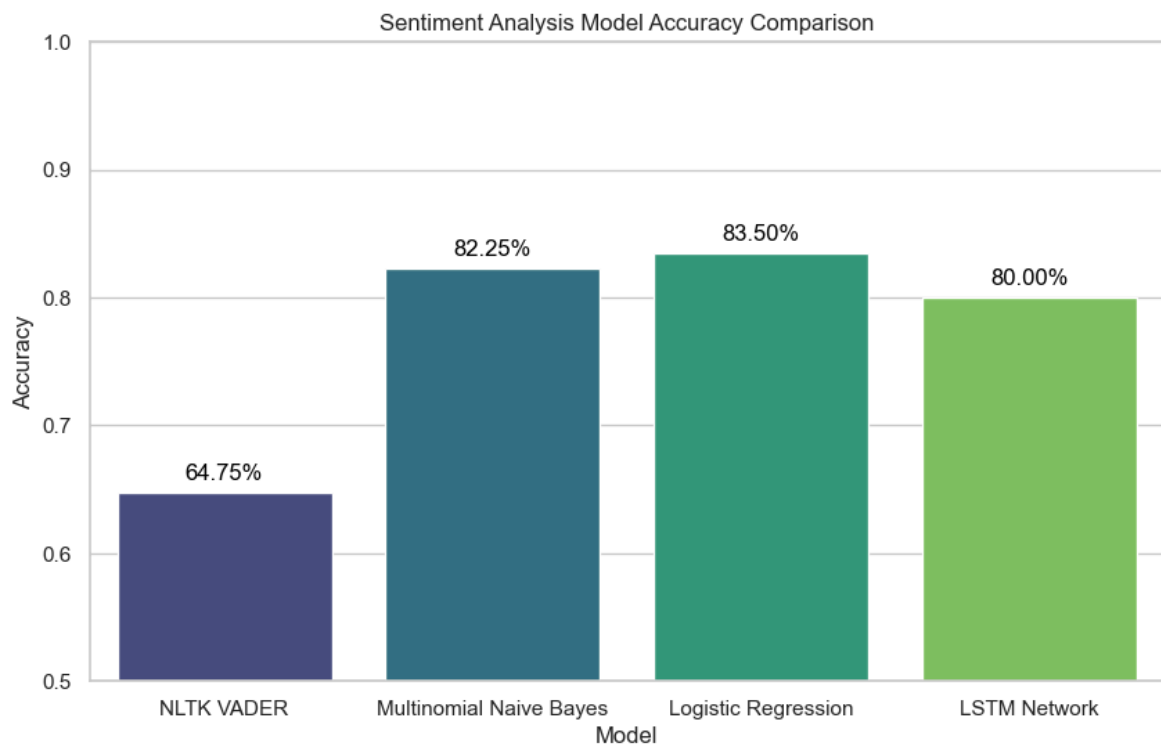
# Plot accuracy comparison
plt.figure(figsize=(10, 6))
sns.barplot(data=comparison_df, x='Model', y='Accuracy', palette='viridis')
plt.title('Sentiment Analysis Model Accuracy Comparison')
plt.ylabel('Accuracy')
plt.ylim(0.5, 1.0)
for index, row in comparison_df.iterrows():
    plt.text(index, row['Accuracy'] + 0.01, f'{row["Accuracy"]:.2%}', color='black')
plt.show()
```

```
--- Model Performance Summary ---
              Model  Accuracy  ROC-AUC
Multinomial Naive Bayes  0.8225  0.901675
Logistic Regression      0.8350  0.920550
LSTM Network             0.8000  0.874425
```

C:\Users\purva\AppData\Local\Temp\ipykernel_42140\3472395946.py:19: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(data=comparison_df, x='Model', y='Accuracy', palette='viridis')
```



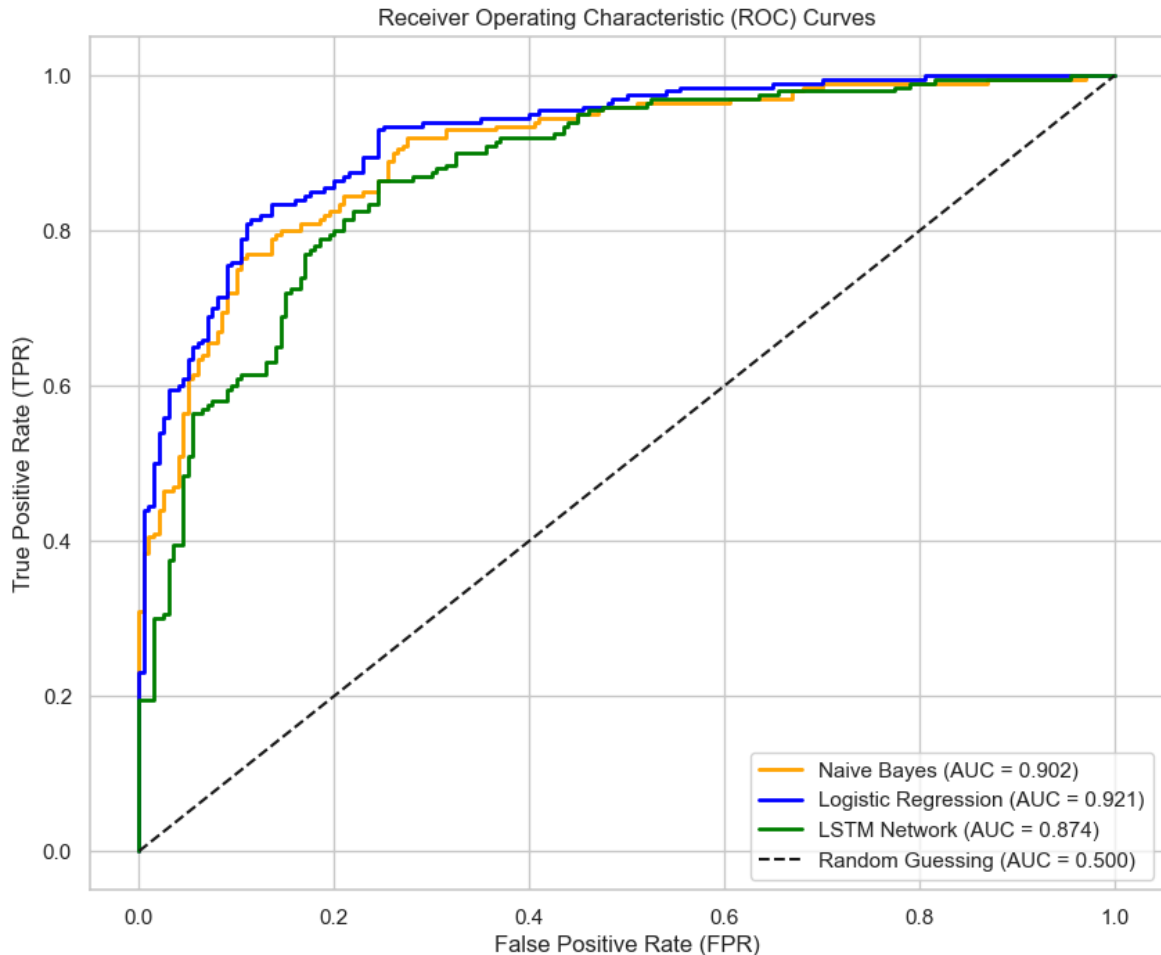
Let's compare the ROC Curves for the models.

```
In [19]: # Compute ROC curves
fpr_nb, tpr_nb, _ = roc_curve(y_test, nb_probs)
fpr_lr, tpr_lr, _ = roc_curve(y_test, lr_probs)
fpr_lstm, tpr_lstm, _ = roc_curve(y_test, lstm_probs)

plt.figure(figsize=(10, 8))
plt.plot(fpr_nb, tpr_nb, label=f'Naive Bayes (AUC = {nb_auc:.3f})', color='orange')
plt.plot(fpr_lr, tpr_lr, label=f'Logistic Regression (AUC = {lr_auc:.3f})', color='blue')
```

```
plt.plot(fpr_lstm, tpr_lstm, label=f'LSTM Network (AUC = {lstm_auc:.3f})', color='green')
plt.plot([0, 1], [0, 1], 'k--', label='Random Guessing (AUC = 0.500)')

plt.title('Receiver Operating Characteristic (ROC) Curves')
plt.xlabel('False Positive Rate (FPR)')
plt.ylabel('True Positive Rate (TPR)')
plt.legend(loc='lower right')
plt.show()
```



We plot the Confusion Matrices to visualize true positives, true negatives, false positives, and false negatives.

```
In [20]: fig, axes = plt.subplots(1, 3, figsize=(20, 5))

# Naive Bayes CM
cm_nb = confusion_matrix(y_test, nb_preds)
sns.heatmap(cm_nb, annot=True, fmt='d', cmap='Blues', cbar=False, ax=axes[0])
axes[0].set_title('Naive Bayes Confusion Matrix')
axes[0].set_xlabel('Predicted Label')
axes[0].set_ylabel('True Label')
axes[0].set_xticklabels(['Negative', 'Positive'])
axes[0].set_yticklabels(['Negative', 'Positive'])

# Logistic Regression CM
cm_lr = confusion_matrix(y_test, lr_preds)
sns.heatmap(cm_lr, annot=True, fmt='d', cmap='Greens', cbar=False, ax=axes[1])
axes[1].set_title('Logistic Regression Confusion Matrix')
axes[1].set_xlabel('Predicted Label')
axes[1].set_ylabel('True Label')
axes[1].set_xticklabels(['Negative', 'Positive'])
```

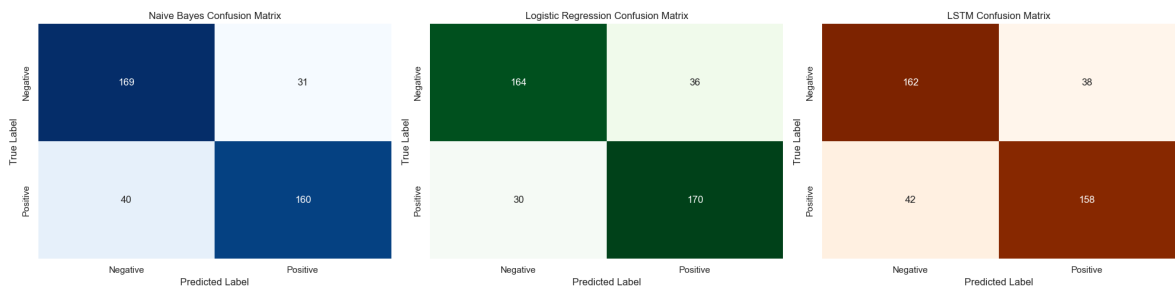
```

axes[1].set_yticklabels(['Negative', 'Positive'])

# LSTM CM
cm_lstm = confusion_matrix(y_test, lstm_preds)
sns.heatmap(cm_lstm, annot=True, fmt='d', cmap='Oranges', cbar=False, ax=axes[2])
axes[2].set_title('LSTM Confusion Matrix')
axes[2].set_xlabel('Predicted Label')
axes[2].set_ylabel('True Label')
axes[2].set_xticklabels(['Negative', 'Positive'])
axes[2].set_yticklabels(['Negative', 'Positive'])

plt.tight_layout()
plt.show()

```



13. Inference Pipeline: Predict Sentiment on Custom Sentences

We build a custom prediction wrapper function to predict the sentiment of unseen test reviews.

```

In [21]: def test_custom_review(text):
# Preprocess text
cleaned_txt = preprocess_text(text)

# 1. TF-IDF vectorization for ML models
tfidf_vec = vectorizer.transform([cleaned_txt])
nb_pred_label = "Positive" if nb_model.predict(tfidf_vec)[0] == 1 else "Negative"
lr_pred_label = "Positive" if lr_model.predict(tfidf_vec)[0] == 1 else "Negative"

# 2. Tokenizer sequence formatting for LSTM
seq = pad_sequences(dl_tokenizer.texts_to_sequences([cleaned_txt]), maxlen=maxlen)
lstm_prob = lstm_model.predict(seq).flatten()[0]
lstm_pred_label = "Positive" if lstm_prob >= 0.5 else "Negative"

# 3. VADER
vader_score = sia.polarity_scores(text)['compound']
vader_pred_label = "Positive" if vader_score >= 0.05 else "Negative"

print(f"Review: \"{text}\"")
print("-" * 50)
print(f"NLTK VADER: {vader_pred_label} (Compound: {vader_score:.4f})")
print(f"Naive Bayes: {nb_pred_label}")
print(f"Logistic Regression: {lr_pred_label}")
print(f"LSTM Deep Learning: {lstm_pred_label} (Confidence: {lstm_prob:.4f})")
print("\n")

# Let's test on custom samples
test_reviews = [

```

```
"This was a masterpiece! Outstanding acting, brilliant cinematography, and a
"What a complete waste of time. The plot was non-existent, the pacing was pa
"It had some decent special effects and a few funny moments, but overall the
```

```
for review in test_reviews:
    test_custom_review(review)
```

1/1 0s 35ms/step

[illegible]

1/1 ————— 0s 55ms/step

Review: "This was a masterpiece! Outstanding acting, brilliant cinematography, and a captivating script. Highly recommend!"

NLTK VADER: Positive (Compound: 0.9458)

Naive Bayes: Positive

Logistic Regression: Positive

LSTM Deep Learning: Negative (Confidence: 0.0127 if positive, 0.9873 if negative)

1/1 0s 29ms/step

□□□

1/1 ————— 0s 49ms/step

Review: "What a complete waste of time. The plot was non-existent, the pacing was painfully slow, and the characters were incredibly flat."

NLTK VADER: Negative (Compound: -0.7351)

Naive Bayes: Negative

Logistic Regression: Negative

LSTM Deep Learning: Negative (Confidence: 0.0053 if positive, 0.9947 if negative)

1/1 ————— 0s 30ms/step

□□□

1/1 ————— 0s 49ms/step

Review: "It had some decent special effects and a few funny moments, but overall the storytelling was weak and the ending was rushed and unsatisfying."

NLTK VADER: Negative (Compound: -0.2617)

Naive Bayes: Positive

Logistic Regression: Positive

LSTM Deep Learning: Negative (Confidence: 0.0256 if positive, 0.9744 if negative)